

Massively による Boltzmann Particle Hydrodynamics のポータブル GPU 実装

Akira Hayakawa

2026 年 7 月 18 日

概要

粒子法は、数値流体力学に対して相補的な計算手段を提供する。Smoothed Particle Hydrodynamics (SPH) は、近傍の物質粒子間のカーネル重み付き相互作用によって連続体方程式を離散化する。一方、Boltzmann Particle Hydrodynamics (BPH) は、速度分布を Monte Carlo 粒子で表現し、自由飛行と直交格子セル内の確率的緩和を交互に行う。BPH は高 Mach 数流れや真空を扱う上で魅力的であるが、標本ゆらぎを抑えるには多数の粒子が必要であり、高いメモリスループットが要求される。本論文では、`massively` v0.80 と CubeCL カーネル言語に基づく Rust 製のポータブル GPU 実装 `bph-gpu` を示す。中心的な実装課題は、緩和がセル局所である一方、セル粒子数が不均一で自由飛行ごとに変化し、更新後の状態は粒子単位で対応を保って書き戻す必要があることである。この不整合を `sort-by-key`、`reduce-by-key`、`scatter`、`gather`、`masked transform`、`stream compaction` の合成によって解決する。これらの操作は、セルごとの逐次ループや競合する `atomic` 加算を使わず、セル局所緩和を規則的な粒子配列とセル配列上の処理へ変換する。緩和更新を導出し、GPU 実装に必要な順序・データ配置の不変条件を示すとともに、再実装可能な粒度で並列プリミティブ列を記述する。演算子テストと Sod 衝撃波管計算を簡潔な実装実証として示す。本論文の焦点は GPU への写像であり、BPH の新たな物理検証や特定 device に対する benchmark 上の主張ではない。

キーワード： Boltzmann Particle Hydrodynamics、Smoothed Particle Hydrodynamics、GPU、Rust、並列プリミティブ、Monte Carlo 粒子法

1 序論

粒子法は数値流体力学において長い歴史を持つ。SPH は天体物理学的な気体力学を対象として、Lucy と Gingold–Monaghan により独立に導入された [1, 2]。SPH は移動する物質粒子によって連続体を表現し、場とその勾配を、近傍粒子上のコンパクト台カーネル和で近似する。この Lagrange 的かつ mesh-free な記述は、大変形、移動界面、自由表面に対して特に有効である [3]。

これと相補的な発展は気体分子運動論から始まる。Direct Simulation Monte Carlo (DSMC) は、分子速度分布を simulation 粒子で表現し、決定論的な移動と確率的衝突を分離する [4]。Isaka、Matsuda、Boffin はこの考えを Molecular Hydrodynamics (MH) として連続体気体流れへ適用し、内部自由度、衝撃波、真空、高 Mach 数状態を扱った [5]。その後 Murata らは、この連続体粒子 scheme の安定性と時間刻み依存性を解析した [6]。日本流体力学会年会講演論文集に発表された後続研究では

BPH という名称が用いられ、可変粒子数および可変重みが検討された [7]。本論文で扱う BPH は同じ stream-relax 構造に従うが、明示的な分子衝突を、Boltzmann 方程式の BGK 近似に基づくセル局所緩和に置き換える [8]。

したがって SPH と BPH はいずれも Lagrange 的粒子法として関連するが、同じ離散化ではない。SPH 粒子は連続体方程式に対する移動求積点として働き、平滑化カーネルを介して相互作用する。BPH 粒子は位相空間分布を標本化し、固定セルが緩和を局所化する。密度、bulk 速度、圧力は粒子集団の moment として復元される。自由飛行では、1 回の更新で BPH 粒子が複数セルを横断できる。このため通常の陽解法格子における CFL 限界を超えても安定な計算が可能である。ただし、大きな時間刻みは分離誤差と数値粘性を通じて精度を低下させる [5, 6]。

粒子表現では、密度が非負の粒子数から、熱エネルギーが非負の運動エネルギーと内部エネルギーから得られるため、極端な状態を扱いやすい。主な代償は Monte Carlo noise である。 N_g 個の粒子を含むセルでは、通常の標本誤差は $O(N_g^{-1/2})$ で減少する。誤差を 10 分の 1 にするには約 100 倍の標本が必要となる。したがって、メモリ容量とスループットが中心的な課題になる。

関連する粒子法の GPU 研究から、有用な計算 pattern が得られている。GPU 高速化 SPH は空間セル、粒子の並べ替え、近傍処理、device 上の時間積分を用いて数百万粒子を扱う [9]。全処理を device 上で行う DSMC 実装も、粒子移動、セル番号計算、衝突、標本化を並列化する [10]。BPH にも同様に data parallelism を利用する余地があるが、そのセル緩和に必要なのは、SPH のカーネル力評価や DSMC の粒子対衝突選択ではなく、segment ごとの集約と再配布である。

本研究では、massively v0.80 と CubeCL を基盤として Rust で `bph-gpu` を開発する [11, 12]。主題は、BPH を GPU 上で実行するために必要なアルゴリズム上の写像である。本論文の貢献は 3 点である。第 1 に、等質量と完全緩和という仮定を含め、並列実装が保存すべき数学的更新を明示する。第 2 に、time step ごとに変わるセル所属、著しく不均一なセル粒子数、粒子データとセルデータの変換、空セル、全粒子場へ同じ並べ替えを適用する必要性、という GPU 固有の課題を整理する。第 3 に、massively の公開並列プリミティブを使い、各課題を device 上の配列操作のポータブルな合成として表現する方法を示す。衝撃波管は、合成した実装が意図した BPH cycle を実行することの実証としてのみ用い、包括的な物理検証とは位置付けない。

2 理論的背景

2.1 BGK 方程式と演算子分割

$\mathbf{x}$ を位置、 $\mathbf{c}$ を分子速度、 $\mathbf{F}$ を外力加速度、 n を数密度、 f を規格化された速度分布とする。BGK 方程式は

$$\frac{\partial(nf)}{\partial t} + \mathbf{c} \cdot \nabla_{\mathbf{x}}(nf) + \mathbf{F} \cdot \nabla_{\mathbf{c}}(nf) = n \frac{f^{\text{eq}} - f}{\tau}, \quad (1)$$

である。ここで τ は緩和時間である [8]。BPH で用いる平衡分布の代替 f^{eq} は Maxwell 分布である必要はない。速度空間で等方的であり、質量、運動量、全エネルギーの moment

$$\int n f Q(\mathbf{c}) d\mathbf{c} = \int n f^{\text{eq}} Q(\mathbf{c}) d\mathbf{c}, \quad Q \in \{m, m\mathbf{c}, \frac{1}{2}m|\mathbf{c}|^2 + e\}. \quad (2)$$

を保存すれば十分である。本論文で調べる実装は、この平衡分布代替の角度成分に球殻分布を用いる。

式 (1) を輸送と衝突に分割する。概念的には、1 time step は

$$\text{streaming: } (\mathbf{x}, \mathbf{c}) \longrightarrow (\mathbf{x}^*, \mathbf{c}^*), \quad \text{relaxation: } f^* \longrightarrow f^{n+1}. \quad (3)$$

となる。数値解法では図 1 に示す順序、すなわちセル番号計算、粒子の並べ替え、緩和、streaming、境界条件の適用を反復する。ある反復で得られた位置を次の反復の緩和が利用するため、この順序と move-then-relax という記述との差は時間 label だけである。外力がない場合、1 次精度の streaming 更新は

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \mathbf{c}^{n+1} \Delta t. \quad (4)$$

である。第 4 節の数値実証では外力を用いない。

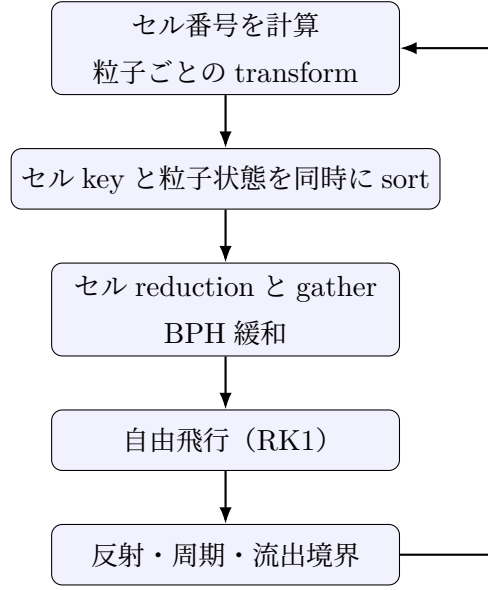


図 1 BPH の 1 time step。GPU 実装では、loop 全体を通じて粒子状態を device 上に保持する。

2.2 粒子状態、比熱比、および前提条件

粒子 i は、位置 $\mathbf{x}_i$ 、速度 $\mathbf{c}_i = (u_i, v_i, w_i)$ 、質量 m_i 、内部エネルギー e_i 、セル番号 $g(i)$ を持つ。3 つの並進自由度に加えて、粒子は $s \geq 0$ 個の内部自由度を持つ。対応する比熱比は

$$\gamma = 1 + \frac{2}{3+s} = \frac{s+5}{s+3}. \quad (5)$$

である。したがって $s = 0$ では $\gamma = 5/3$ 、 $s = 2$ では $\gamma = 7/5$ となる。

ここで扱う定式化では、1 つの緩和に参加する粒子の質量と s が等しいと仮定する。粒子質量が異なる場合、式 (6) の算術平均を質量重み付き平均に置き換える必要がある。したがって、以下の保存則の導出と数値実証では $m_i = m$ と仮定する。

2.3 単一セル内の緩和

I_g をセル g に属する粒子の集合、 $N_g = |I_g|$ とする。 $N_g < 2$ のセルでは衝突を model 化できないため、状態を変更しない。 $N_g \geq 2$ の場合、平均速度と熱速度を

$$\mathbf{U}_g = \frac{1}{N_g} \sum_{i \in I_g} \mathbf{c}_i, \quad \mathbf{q}_i = \mathbf{c}_i - \mathbf{U}_g. \quad (6)$$

と定義する。重心系で再配分されるエネルギーは

$$E_g = \sum_{i \in I_g} \left(\frac{1}{2} m |\mathbf{q}_i|^2 + e_i \right). \quad (7)$$

である。エネルギー等分配則により、緩和後の並進運動エネルギーと内部エネルギーを $3:s$ の比で

$$K_g^* = \frac{3}{3+s} E_g, \quad E_{\text{int},g}^* = \frac{s}{3+s} E_g. \quad (8)$$

に配分する。各粒子について、2つの独立な一様乱数 $r_{1i}, r_{2i} \in [0, 1)$ から

$$\mu_i = 1 - 2r_{1i}, \quad \theta_i = 2\pi r_{2i}, \quad (9)$$

$$\mathbf{d}_i = \left(\sqrt{1 - \mu_i^2} \sin \theta_i, \sqrt{1 - \mu_i^2} \cos \theta_i, \mu_i \right). \quad (10)$$

を生成する。 $|\mathbf{d}_i| = 1$ であるため、これらの vector は単位球面上に一様分布する。有限標本の vector 和は厳密には 0 にならないので、セルごとに

$$\hat{\mathbf{d}}_i = \mathbf{d}_i - \frac{1}{N_g} \sum_{j \in I_g} \mathbf{d}_j, \quad K_g^{\text{raw}} = \sum_{i \in I_g} \frac{1}{2} m |\hat{\mathbf{d}}_i|^2. \quad (11)$$

として balance を取る。速度 scale を

$$a_g = \sqrt{\frac{K_g^*}{K_g^{\text{raw}}}}, \quad (12)$$

とすると、緩和後の状態は

$$\mathbf{c}'_i = \mathbf{U}_g + a_g \hat{\mathbf{d}}_i, \quad e'_i = \frac{E_{\text{int},g}^*}{N_g}. \quad (13)$$

となる。式 (12) による再 scale には $K_g^{\text{raw}} > 0$ が必要である。連続分布から独立に方向を標本化する場合、 $N_g \geq 2$ でこの退化事象が起こる確率は 0 であるが、有限の乱数生成器では表現され得る。実装は除算を guard して scale を 0 にする。この例外でも保存を要求する再実装では、方向を再標本化するか、和が 0 となる決定論的な球殻方向を fallback として与える必要がある。

2.4 保存則と数値的性質

式 (11) より $\sum_i \hat{\mathbf{d}}_i = \mathbf{0}$ である。したがって

$$\sum_{i \in I_g} m \mathbf{c}'_i = m N_g \mathbf{U}_g = \sum_{i \in I_g} m \mathbf{c}_i. \quad (14)$$

となり、セルの運動量が保存される。交差項も消えるため、

$$\sum_{i \in I_g} \left(\frac{1}{2} m |c'_i|^2 + e'_i \right) = \frac{1}{2} m N_g |U_g|^2 + K_g^* + E_{\text{int},g}^* \quad (15)$$

$$= \frac{1}{2} m N_g |U_g|^2 + E_g. \quad (16)$$

となる。よって $K_g^{\text{raw}} > 0$ かつ厳密演算では全エネルギーも保存される。セル緩和は粒子を生成も除去もしないため、質量保存は自明である。ただし、流出境界では計算領域内の質量が減少し得る。

BGK 衝突項を forward Euler 離散化すると、緩和確率 $P = \Delta t / \tau$ を導入できる。粘性流では、この確率で緩和を標本化することにより有限の平均自由行程を表現できる。一方、 $P = 1$ とすると、現在の `bph-gpu` コードが実装する完全緩和 Euler model になる。有限 P の Navier–Stokes 粘性 model はまだ実装していない。

BPH が CFL 条件に制限されないという性質は、粒子がセル境界を横断できることに由来する。これは、任意の時間刻みで高精度であることを意味しない。 Δt を増やすと分離誤差と数値粘性が増え、境界処理では長い自由飛行中のすべての横断を整合的に扱う必要がある。

3 Massively による GPU への写像

3.1 セル緩和を GPU 化する際の課題

自由飛行は粒子局所処理であるが、緩和は純粋な粒子局所 kernel でも固定幅のセル stencil でもない。各粒子は最初に現在の所属セルを求める必要がある。1 セルの粒子数は 0 から全粒子の大きな割合まで変化し得る上、その分布は time step ごとに変わる。したがって、各セルに 1 thread または固定 workgroup を割り当てると、負荷不均衡と必要 local memory 量が事前に分からないという問題が生じる。一方、各粒子に 1 thread を割り当てただけでは、多対一の集約を行わない限り、セル和、平均、エネルギー scale を得られない。

このため実装は、2 つの index 空間を交互に扱う必要がある。長さ N の粒子空間は、streaming、乱数標本化、最終状態更新に適する。長さ K のセル空間は、粒子数、速度和、エネルギー、緩和 scale を保持する。実用的な GPU アルゴリズムでは、粒子値をセルへ集約し、空セルを表現し、セル値を粒子へ再配布すると同時に、structure-of-arrays の全列で粒子の対応を維持しなければならない。表 1 に、massively v0.80 によるこれらの変換をまとめる。

表 1 BPH の GPU 化における課題と `massively` v0.80 による実現。

BPH の要件	GPU 上の課題	Massively による合成
現在のセル群を形成	streaming 後に所属が変化	1 つの共有置換による <code>sort_by_key</code>
セル moment を計算	不均一な多対一集約	同一 key 連続区間上の <code>reduce_by_key</code>
空セルを含める	reduction は非空 key だけを出力	0 初期化セル配列と <code>scatter</code>
セル値を粒子へ適用	結果がセル index 空間に存在	key による <code>gather</code> または等価な 遅延 index read
衝突可能セルだけ更新	粒子数 2 未満のセルは不変	<code>masked transform</code> または <code>gather_where</code>
流出粒子を除去	粒子数が動的に変化	境界述語による <code>remove_where</code>

3.2 実行 model とデータ配置

`bph-gpu` は、`massively` v0.80 並列アルゴリズムライブラリと CubeCL カーネル言語を用いて BPH を Rust で実装する [11, 12]。時間積分 loop の全体を通じて、粒子データを device memory に保持する。runtime 抽象化により、同じアルゴリズムを WGPU、CUDA、HIP へ lowering できる。本論文の数値結果には WGPU を用い、backend 間の等価性と性能の測定は今後の課題とする。

粒子状態 x, y, z, u, v, w, e は、それぞれ独立した連続配列に格納する。これは structure-of-arrays (SoA) 配置である。粒子を並べ替えるすべての処理では、1 つの index 置換を計算し、質量が大域定数でない場合は質量も含め、すべての状態列に同じ置換を適用する。これにより各物理量の列は連続のままとなり、粒子ごとの coalesced access が可能になる。粒子配列とセル key 配列は、常に同じ長さで粒子順序を持たなければならない。

セル配列は空セルの entry も含め、常に長さ K とする。reduce-by-key は非空セルに対する結果だけを出力するため、この compact な出力をセル番号に従って、0 初期化した dense 配列へ scatter する。次に gather によって、各セルの値を所属粒子へ配布する。この reduction–scatter–gather cycle が、セル局所物理量と粒子 level の並列処理を結び付ける。

3.3 セル key と順序の不変条件

粒子数を N 、直交格子のセル数を $K = n_x n_y n_z$ とする。セル幅を h_α 、計算領域の下端座標を $x_{\alpha, \min}$ とすると、領域内部の粒子に対するセル番号は

$$j_{\alpha i} = \left\lfloor \frac{x_{\alpha i} - x_{\alpha, \min}}{h_\alpha} \right\rfloor, \quad 0 \leq j_{\alpha i} < n_\alpha, \quad (17)$$

$$g_i = j_{xi} + n_x (j_{yi} + n_y j_{zi}), \quad 0 \leq g_i < K. \quad (18)$$

となる。式 (18) を次に評価する前に、周期境界または反射境界を越えた座標を領域内へ戻し、流出粒子を除去しなければならない。前回の自由飛行 step で得た key を再利用してはならない。

key–state 対を g_i の昇順に sort し、得られた置換をすべての粒子場へ適用する。同じ key 間の stable 性は物理的要件ではない。必要な不変条件は、同一 key が連続区間を形成し、すべての場が粒

子との対応を維持することである。空間 sort は GPU 上の SPH および DSMC 実装でも標準的な構成要素である [9, 10]。

3.4 Massively プリミティブの合成

表 1 の名称は、massively v0.80 が公開するアルゴリズム interface、すなわち `sort_by_key`、`reduce_by_key`、`scatter`、`gather`、`lower_bound`、`upper_bound`、`masked transform` に対応する。重要なのはこれらの合成であり、ライブラリ固有の iterator 型や整数変換型ではない。同じ意味の操作を提供する別の GPU framework でも、同じ分解を再現できる。

順序を明示する 1 つの方法は、対 (g_i, i) を第 1 成分で sort することである。得られた置換を p とすると、 $g_{p_0} \leq g_{p_1} \leq \dots \leq g_{p_{N-1}}$ となり、すべての粒子場を p の順序で gather する。すべての状態場へ厳密に同じ置換を適用するなら、key-state record を直接 sort しても等価である。順序は各 time step に 1 回だけ計算し、全場で共有すべきである。

中心となる再利用可能な処理は dense cell reduction である。sort 済み key $\mathbf{g}$ と粒子値 $\mathbf{a}$ を与えると、reduce-by-key はまず、非空セルの一意な key κ と compact な和 $\mathbf{s}$ を返す。これらを 0 初期化したセル配列へ scatter すると、

$$(\kappa, \mathbf{s}) = \text{reduce_by_key}(\mathbf{g}, \mathbf{a}), \quad \mathbf{A} = \mathbf{0}_K, \quad A_{\kappa_j} \leftarrow s_j. \quad (19)$$

となる。この構成は、競合する粒子-セル間 atomic 加算を使わずに、空セルを明示的に表現する。セル g の粒子数は、sort 済み key 配列における g の upper bound と lower bound の差である。これと等価に、全要素が 1 の粒子配列へ式 (19) を適用しても求められる。

任意の dense セル場 A_g に対し、粒子 key による置換から粒子場 $a_i = A_{g_i}$ を構成する。GPU 実装では、この index read を消費側 kernel へ融合しても、gather 後の粒子配列を実体化してもよい。両者の数学的意味は同じである。SoA 場の対応を維持する限り、 (u, v, w) のような vector 量は成分ごとに処理しても tuple として処理してもよい。 $N_g \geq 2$ という mask で緩和対象粒子を選び、それより小さいセルの粒子は以前の状態を保持する。

3.5 並列緩和

以下の手順により、完全緩和 BPH の衝突 step を再実装できる。ここで DENSEREDUCE は式 (19)、 $\text{GATHER}(\mathbf{A}, \mathbf{g})_i = A_{g_i}$ を表し、すべての粒子 map は並列である。

1. N_g と $\mathbf{S}_g = \text{DENSEREDUCE}(\mathbf{g}, \mathbf{c})_g$ を計算し、非空セルについて $\mathbf{U}_g = \mathbf{S}_g / N_g$ を求める。
2. $\mathbf{U}_{g_i}$ を gather して緩和の最後まで保存し、各速度を中心化する： $\mathbf{q}_i = \mathbf{c}_i - \mathbf{U}_{g_i}$ 。
3. 次を計算する： $E_g = \text{DENSEREDUCE}(\mathbf{g}, \frac{1}{2}m|\mathbf{q}|^2 + e)_g$ 。
4. 2 つの独立な一様乱数粒子配列を生成し、式 (10) で transform して $\mathbf{d}_i$ を得る。
5. 次を計算する： $\bar{\mathbf{d}}_g = \text{DENSEREDUCE}(\mathbf{g}, \mathbf{d})_g / N_g$ 。さらに $\hat{\mathbf{d}}_i = \mathbf{d}_i - \bar{\mathbf{d}}_{g_i}$ 。期待値としての等方性だけでなく、この有限標本補正によって離散的な運動量保存を保証する。
6. 次を計算する： $K_g^{\text{raw}} = \text{DENSEREDUCE}(\mathbf{g}, \frac{1}{2}m|\hat{\mathbf{d}}|^2)_g$ 。次に式 (8) および (12) から K_g^* 、 $E_{\text{int},g}^*$ 、 a_g を求める。
7. $N_g \geq 2$ のセルについて、これらのセル値を gather し、 $\mathbf{c}'_i = \mathbf{U}_{g_i} + a_{g_i}\hat{\mathbf{d}}_i$ および $e'_i = E_{\text{int},g_i}^* / N_{g_i}$ を書き込む。 $N_g < 2$ のセル内粒子は変更せず、平方根の前に $K_g^{\text{raw}} = 0$ を guard する。

緩和後、式 (4) によって位置を進め、選択した境界演算子を適用する。次の step ではセル key を再計算して sort する。2 つの角度乱数には独立な乱数 stream が必要であり、time step 間で stream を変えることで同じ角度標本の反復を避ける。再現可能なテストには決定論的な seed schedule が有用である。ただし、乱数生成器と浮動小数点 reduction 順序が異なり得るため、backend 間の bit 単位一致は仮定しない。

3.6 メモリ転送量と性能上の考察

粒子単位の並列化により、粒子数が 0 から数千まで変化するセルに 1 thread を割り当てる場合の負荷不均衡を避けられる。ただし、大域的なデータ移動がなくなるわけではない。各 time step では全粒子を走査する pass を複数回実行し、sort ではすべての動的粒子列へ置換を適用する。比較 sort の仕事量は $O(N \log N)$ であり、緩和には複数の reduction と粒子-セル写像が必要である。したがって、これらが profiling の主対象となる。ただし本実装の測定なしに、処理時間の内訳は主張しない。

index 付き read を消費側 kernel へ融合すると一部の間接配列を省略できるが、いくつかの段階では依然として N または K 要素の一時 buffer が必要である。したがって、kernel fusion、buffer の永続的再利用、有界整数セル sort、および sort を用いない集約が主な最適化候補になる。漸近的な追加記憶量は $O(N + K)$ である。

4 実装実証

実装は 2 つの level で動作確認する。演算子テストでは、segment reduction が空セルの 0 を含む所定の dense セル和を生成すること、1 つの共有 sort 置換が全粒子列の対応を維持すること、gather したセル平均速度の減算と復元によって元の速度が得られること、および非退化球殻標本について緩和の合成が意図したエネルギー配分を保つことを確認する。これらは GPU への写像が依存するデータフロー上の不変条件を対象とする。

end-to-end の簡潔な実証として Sod 衝撃波管 [13] を用いる。単位区間を $K = 2,000$ セルに分割し、 $\Delta t = \Delta x = 5 \times 10^{-4}$ とする。初期左右状態の密度比は 8 : 1、圧力比は 10 : 1、bulk 速度は 0 である。左右をそれぞれ 1 セル当たり 4,000 粒子と 500 粒子の等質量粒子で表現し、全粒子数は $N = 4,500,000$ となる。 $s = 2$ ($\gamma = 7/5$)、完全緩和、単精度状態、WGPU backend を用い、 $t = 0.15$ まで計算する。密度は各セルの粒子数を左初期状態の 4,000 粒子で除して示す。

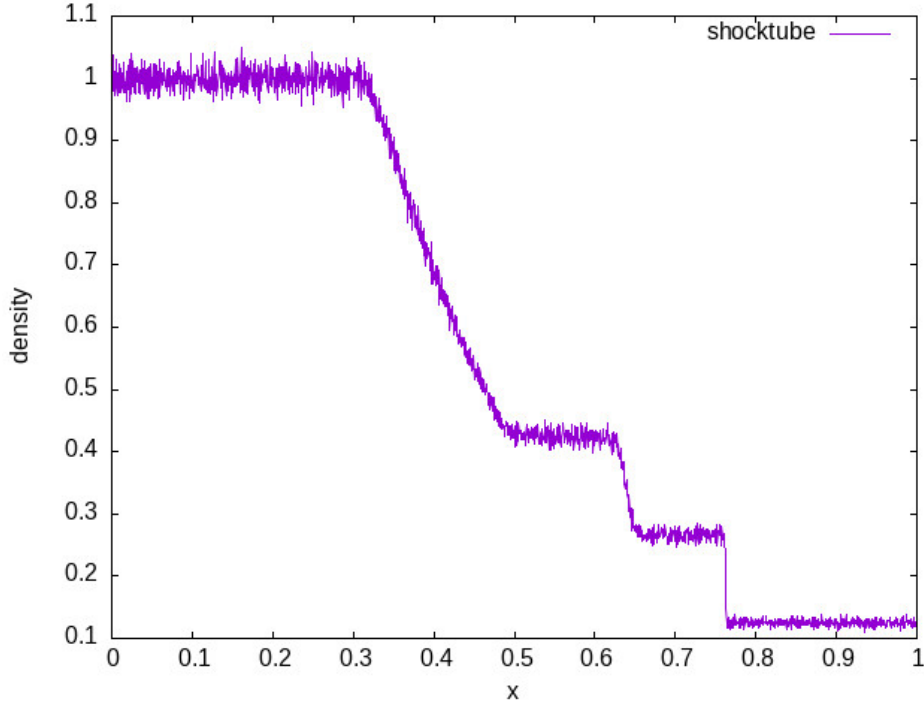


図 2 bph-gpu で計算した Sod 衝撃波管の規格化セル密度。Monte Carlo 粒子密度に、希薄波、接触不連続、衝撃波が現れている。

図 2 の目的は限定的である。セル key 生成、共有並べ替え、segment 緩和、streaming、境界処理を合成し、数百万粒子規模の一連の計算を実行できることを確認する。本テクニカルペーパーでは、この図から収束回数、誤差上界、他実装に対する高速化を主張しない。同様に、backend 間の性能測定も現在の範囲外とする。

5 結論

BPH の GPU 実装における中心的な障害は、粒子局所の実行 model と、時間的に変化する不均一なセル群上の緩和との不整合である。bph-gpu は、massively v0.80 を用いてこれを解決する。sort-by-key はセル segment を連続化し、共有置換によって SoA の対応を維持する。reduce-by-key と scatter は dense セル moment を構成し、gather はそれらを粒子へ戻す。masked transform と compaction は衝突・境界述語を実装する。この構成では、セル単位の逐次 loop も、粒子からセルへの atomic 加算も必要ない。

緩和の導出は、このデータフローが保存すべき量を明らかにし、プリミティブ level の手順は補助的なライブラリ型に依存せずアルゴリズムを規定する。演算子テストによって順序と集約の不変条件を確認し、450 万粒子の Sod 計算によって GPU 上に保持された一連の cycle を実証した。本テクニカルペーパーの意図的に限定した実証範囲において、得られた結果は、CubeCL を通じて複数の GPU runtime family へ lowering 可能な、再実装可能な BPH 並列分解である。

参考文献

- [1] L. B. Lucy, “A Numerical Approach to the Testing of the Fission Hypothesis,” *The Astronomical Journal*, Vol. 82, pp. 1013–1024, 1977. <https://doi.org/10.1086/112164>
- [2] R. A. Gingold and J. J. Monaghan, “Smoothed Particle Hydrodynamics: Theory and Application to Non-Spherical Stars,” *Monthly Notices of the Royal Astronomical Society*, Vol. 181, No. 3, pp. 375–389, 1977. <https://doi.org/10.1093/mnras/181.3.375>
- [3] J. J. Monaghan, “Smoothed Particle Hydrodynamics,” *Annual Review of Astronomy and Astrophysics*, Vol. 30, pp. 543–574, 1992. <https://doi.org/10.1146/annurev.aa.30.090192.002551>
- [4] G. A. Bird, *Molecular Gas Dynamics and the Direct Simulation of Gas Flows*. Oxford University Press, 1994.
- [5] H. Isaka, T. Matsuda, and H. M. J. Boffin, “Application of Molecular Hydrodynamics to Astrophysical Flows: Bondi–Hoyle–Lyttleton Accretion Flow,” *Progress of Theoretical Physics*, Vol. 116, No. 6, pp. 1069–1096, 2006. <https://doi.org/10.1143/PTP.116.1069>
- [6] H. Murata, T. Matsuda, H. Isaka, Y. Ohsugi, and H. M. J. Boffin, “Application of Molecular Hydrodynamics to Astrophysical Flows. II: Unconditional Stability,” *Progress of Theoretical Physics*, Vol. 120, No. 2, pp. 347–369, 2008. <https://doi.org/10.1143/PTP.120.347>
- [7] H. Isaka, T. Matsuda, and Y. Ohsugi, “Improved Boltzmann Particle Method: Variable Number of Particles,” *Proceedings of the Japan Society of Fluid Mechanics Annual Meeting 2010*, p. 204, 2010 (in Japanese). <https://cir.nii.ac.jp/crid/2051996266952625024>
- [8] P. L. Bhatnagar, E. P. Gross, and M. Krook, “A Model for Collision Processes in Gases. I. Small Amplitude Processes in Charged and Neutral One-Component Systems,” *Physical Review*, Vol. 94, pp. 511–525, 1954. <https://doi.org/10.1103/PhysRev.94.511>
- [9] A. J. C. Crespo, J. M. Dominguez, A. Barreiro, M. Gómez-Gesteira, and B. D. Rogers, “GPUs, a New Tool of Acceleration in CFD: Efficiency and Reliability on Smoothed Particle Hydrodynamics Methods,” *PLOS ONE*, Vol. 6, No. 6, e20685, 2011. <https://doi.org/10.1371/journal.pone.0020685>
- [10] C.-C. Su, M. R. Smith, F.-A. Kuo, J.-S. Wu, C.-W. Hsieh, and K.-C. Tseng, “Large-Scale Simulations on Multiple Graphics Processing Units (GPUs) for the Direct Simulation Monte Carlo Method,” *Journal of Computational Physics*, Vol. 231, No. 23, pp. 7932–7958, 2012. <https://doi.org/10.1016/j.jcp.2012.07.038>
- [11] A. Hayakawa, “massively: Multi-platform GPU parallel algorithms for Rust,” version 0.80, software library, 2026. <https://github.com/akiradeveloper/massively>
- [12] Tracel AI, “CubeCL: Multi-platform high-performance computing language extension for Rust,” source code. <https://github.com/tracel-ai/cubec1>
- [13] G. A. Sod, “A Survey of Several Finite Difference Methods for Systems of Nonlinear Hyperbolic Conservation Laws,” *Journal of Computational Physics*, Vol. 27, No. 1, pp. 1–31, 1978. [https://doi.org/10.1016/0021-9991\(78\)90023-2](https://doi.org/10.1016/0021-9991(78)90023-2)